

特征人脸识别-实训报告

2312966 林晖鹏

一、问题重述

题目背景：

本实验采用特征脸（Eigenface）算法进行人脸识别。

特征脸（eigenface）是第一种有效的人脸识别方法，通过在一大组描述不同人脸的图像上进行主成分分析（PCA）获得。

作业要求：

- 基于所提供的数据集，构建特征人脸。
- 用特征人脸进行人脸识别和重建。
- 构建自己的数据集，添加样本。

二、设计思想及主体代码内容

1. 特征人脸算法

在这个算法中，我们先把 $112 * 92$ 的二维图展开为一维向量，然后希望用PCA算法去对这个一维向量降维，降维时用的映射矩阵就是我们构建的特征脸。

算法流程：

- a. 样本归一化
- b. 样本去中心化，即减去所有向量的均值
- c. 计算数据的协方差矩阵
- d. 根据特征值大小，挑选前k的特征向量，得到映射矩阵

但是由于数据太大，大小为 $1 * 10304$ ，在计算协方差矩阵的时候计算量太大了，所以我们不直接求协方差矩阵，而是用SVD奇异值分解去间接求解协方差矩阵，能减少计算量。分解出来的Vt就是协方

差。

代码块

```
1      # 1.求平均人脸
2      avg_img=np.mean(trainset, axis=0)
3
4      # 2.去中心化
5      norm_img = trainset - avg_img
6
7      # 3.奇异值分解
8      U, S, Vt = np.linalg.svd(norm_img, full_matrices=False)
9
10     #4.取前k个特征向量作为特征脸
11     feature = Vt[:k]
```

2. 人脸识别模型

在得到特征人脸空间之后，我们用这些特征人脸进行人脸识别：

我们分别计算测试集和训练集中来自同一个人的图像经过映射矩阵之后的特征向量，计算向量之间的相似度

计算特征向量：

代码块

```
1      # 1.计算特征脸的数量
2      if numComponents == 0:
3          numEigenFaces = eigenface_vects.shape[0]
4      else:
5          numEigenFaces = min(numComponents, eigenface_vects.shape[0])
6
7      eigenface_vects=eigenface_vects[:numEigenFaces]
8      # 2.计算输入数据的特征向量表示
9      representation = np.dot(image - avg_img, eigenface_vects.T)
```

计算相似度：

代码块

```
1      train_reps = []
2      for img in train_vectors:
3          train_rep, _ = rep_face(img, avg_img, eigenface_vects, num_eigenface)
4          train_reps.append(train_rep)
```

```

5
6 num = 0
7 for idx, image in enumerate(test_vectors):
8     label = test_labels[idx]
9     test_rep, _ = rep_face(image, avg_img, eigenface_vects, num_eigenface)
10
11     results = []
12     for train_rep in train_reps:
13         similarity = np.sum(np.square(train_rep - test_rep))
14         results.append(similarity)
15     results = np.array(results)
16
17     if label == np.argmin(results) // 5 + 1:
18
19         num = num + 1
20
21 print("人脸识别准确率: {}".format(num / 100 * 100))

```

3. 人脸重建模型

我们用降维之后的特征向量乘上特征人脸，在加上人脸向量的平均值，得到重建的人脸。这里重建的人脸的质量和选取的特征人脸数有关。

代码块

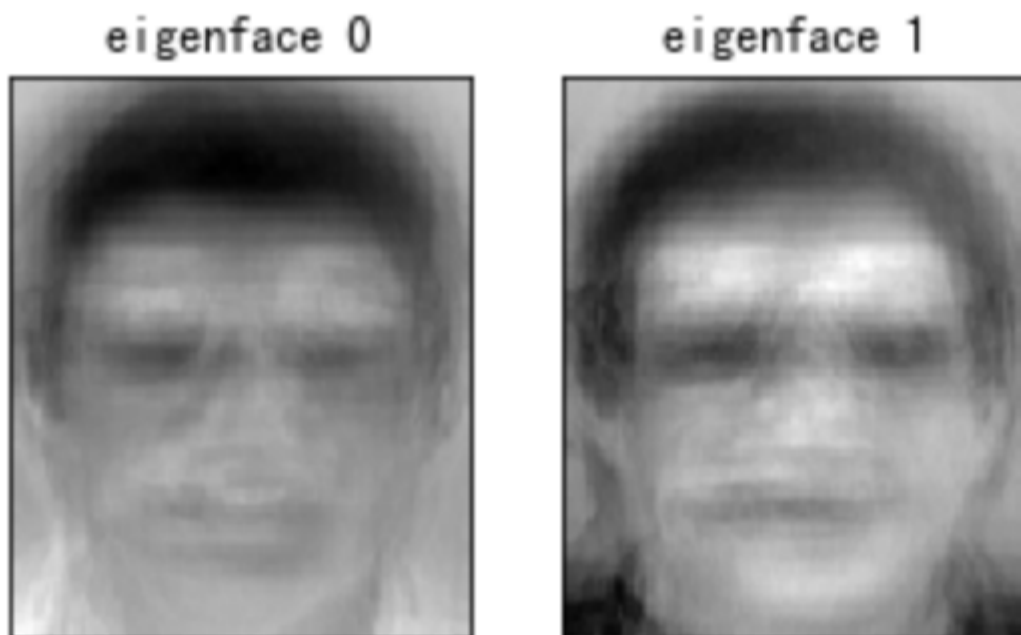
```

1 numComponents = min(numComponents, eigenVectors.shape[0])
2 if numComponents == 0:
3     numComponents = eigenVectors.shape[0]
4 eigenVectors = eigenVectors[:numComponents]
5
6 face_flat = np.dot(representations, eigenVectors)
7 face_flat += avg_img
8 face = face_flat.reshape(sz)

```

三、实验结果

下图是其中两个生成的特征人脸。特征人脸是比较模糊的。



得到人脸识别的准确率为 **91.25 %**

下图是不同特征人脸数重建人脸的效果。可以看到，特征人脸选取越多，图片越清晰，细节也

numEigenFaces_20



numEigenFaces_40



numEigenFaces_60



numEigenFaces_80



numEigenFaces_100



numEigenFaces_120



numEigenFaces_140



numEigenFaces_160



numEigenFaces_180



测试通过!

main.py

接口测试

✓ 接口测试通过。

用例测试

测试点	状态	时长	结果
测试结果	✓	3s	测试成功!

提交结果

四、构建自己的数据库

接下来我们尝试去添加样本进数据集中。

寻找的数据集来自MIT的人脸识别数据集<http://cbcl.mit.edu/software-datasets/heisele/facerecognition-database.html>。包含了十个样本的多张图片。对于每个样本，我们选取7张图片，保持和原数据集一致，然后尽量寻找和原本数据集角度类似的图片。（和原数据集一样的同样本下多视角的数据集太少了）

然后为了满足人脸库中的每一张图片都只包含一张人脸且眼睛的中心位置对齐(通过裁剪或缩放，使得每张人脸图像大小尺寸一致且人脸眼睛的中心位置对齐)的要求，我们用opencv相关库对其裁剪并转换格式为png：

代码块

```
1  import os
2  import cv2
3  import dlib
4  import numpy as np
5  from tqdm import tqdm
6
7  # 初始化dlib
8  detector = dlib.get_frontal_face_detector()
9  predictor = dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")
10
11 def precise_align_face(img, target_size=(92, 112)):
```

```

12     """精确对齐人脸并居中"""
13     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
14     faces = detector(gray)
15
16     if len(faces) != 1:
17         return None
18
19     landmarks = predictor(gray, faces[0])
20
21     # 获取双眼中心坐标
22     left_eye = np.mean([(landmarks.part(i).x, landmarks.part(i).y) for i in
23 range(36, 42)], axis=0)
24     right_eye = np.mean([(landmarks.part(i).x, landmarks.part(i).y) for i in
25 range(42, 48)], axis=0)
26
27     eyes_center = (int((left_eye[0] + right_eye[0]) // 2),
28                    int((left_eye[1] + right_eye[1]) // 2))
29
30     # 计算需要的平移量 (使眼睛中心到目标位置)
31     target_eye_x = target_size[0] // 2 # 水平居中
32     target_eye_y = target_size[1] // 3 # 垂直方向1/3处
33     dx = target_eye_x - eyes_center[0]
34     dy = target_eye_y - eyes_center[1]
35
36     # 创建平移矩阵
37     M = np.float32([[1, 0, dx], [0, 1, dy]])
38     translated = cv2.warpAffine(img, M, (target_size[0], target_size[1]))
39
40     return translated
41
42 # 配置
43 input_folder = "train7"
44 output_folder = "aligned_faces_centered"
45 target_size = (92, 112) # 宽92, 高112
46 os.makedirs(output_folder, exist_ok=True)
47
48 # 初始化数据结构
49 data_array = np.zeros((10, 7, 112, 92), dtype=np.uint8)
50 label_vector = np.zeros((10, 7), dtype=np.int32) + 40
51
52 for person_id in range(10):
53     prefix = f"{person_id:04d}"
54     person_files = [f for f in os.listdir(input_folder)
55                     if f.startswith(prefix) and f.lower().endswith('.png')]
56
57     for sample_idx, filename in enumerate(person_files[:7]): # 每人最多7张
58         img_path = os.path.join(input_folder, filename)
59         img = cv2.imread(img_path)

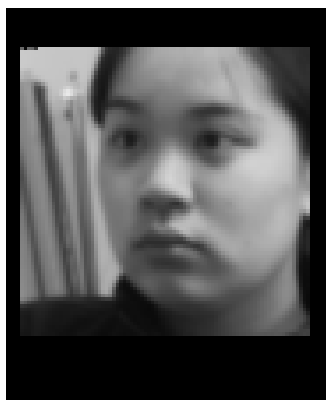
```

```

57         if img is None:
58             continue
59
60         # 精确对齐并居中
61         aligned = precise_align_face(img, target_size)
62         if aligned is None:
63             print(f"无法对齐 {filename}")
64             continue
65
66         gray = cv2.cvtColor(aligned, cv2.COLOR_BGR2GRAY)
67
68         # 保存和存储
69         cv2.imwrite(os.path.join(output_folder, filename), gray)
70         data_array[person_id, sample_idx] = gray
71         label_vector[person_id, sample_idx] = 40 + person_id
72
73
74     np.save(os.path.join(output_folder, "face_data_centered.npy"), data_array )
75
76     np.save(os.path.join(output_folder, "labels_centered.npy"), label_vector)

```

但是加入的数据集并不贴合要求，裁剪之后效果并不好，可以看到人脸还是不在中间，并且由于裁剪，旁边的变成黑边。



然后在拼接到原本的数据当中，并修改相关参数，然后运行原代码：

代码块

```

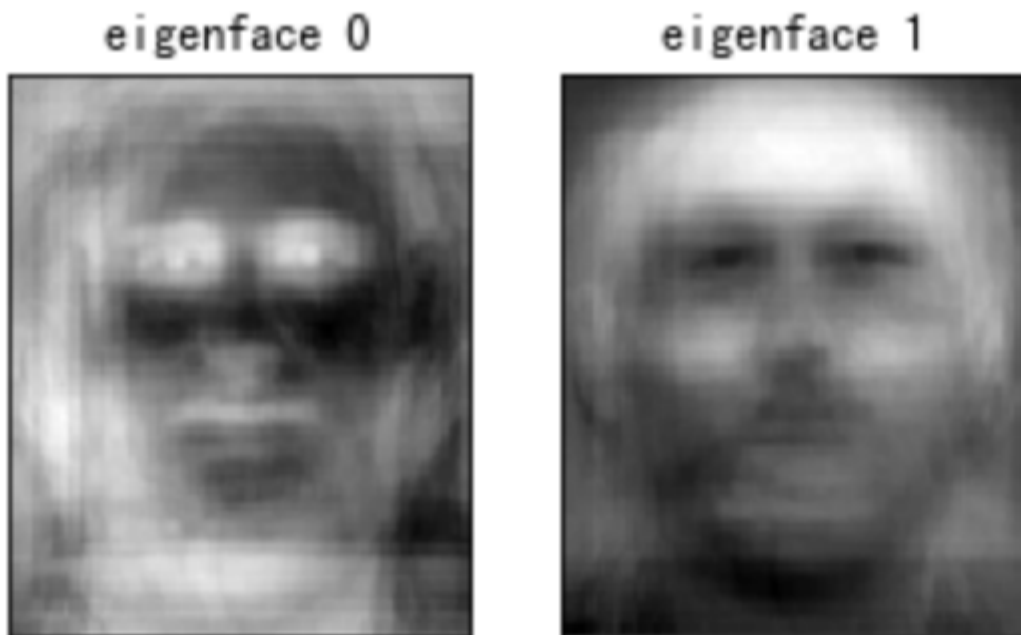
1  datapath = './ORL.npz'
2  ORL = np.load(datapath)
3  data = ORL['data']
4  label = ORL['label']
5  num_eigenface = 250
6
7  data = np.concatenate((data, data_array), axis=0)
8  label = np.concatenate((label, label_vector), axis=0)
9  # print(label)

```

```
10
11
12 train_vectors, train_labels, test_vectors, test_labels = spilt_data(50, 5,
    data, label)
13 # 这里除以255是为了归一化
14 train_vectors = train_vectors / 255
15 test_vectors = test_vectors / 255
16
17 print("训练数据集:", train_vectors.shape)
18 print("测试数据集:", test_vectors.shape)
19
```

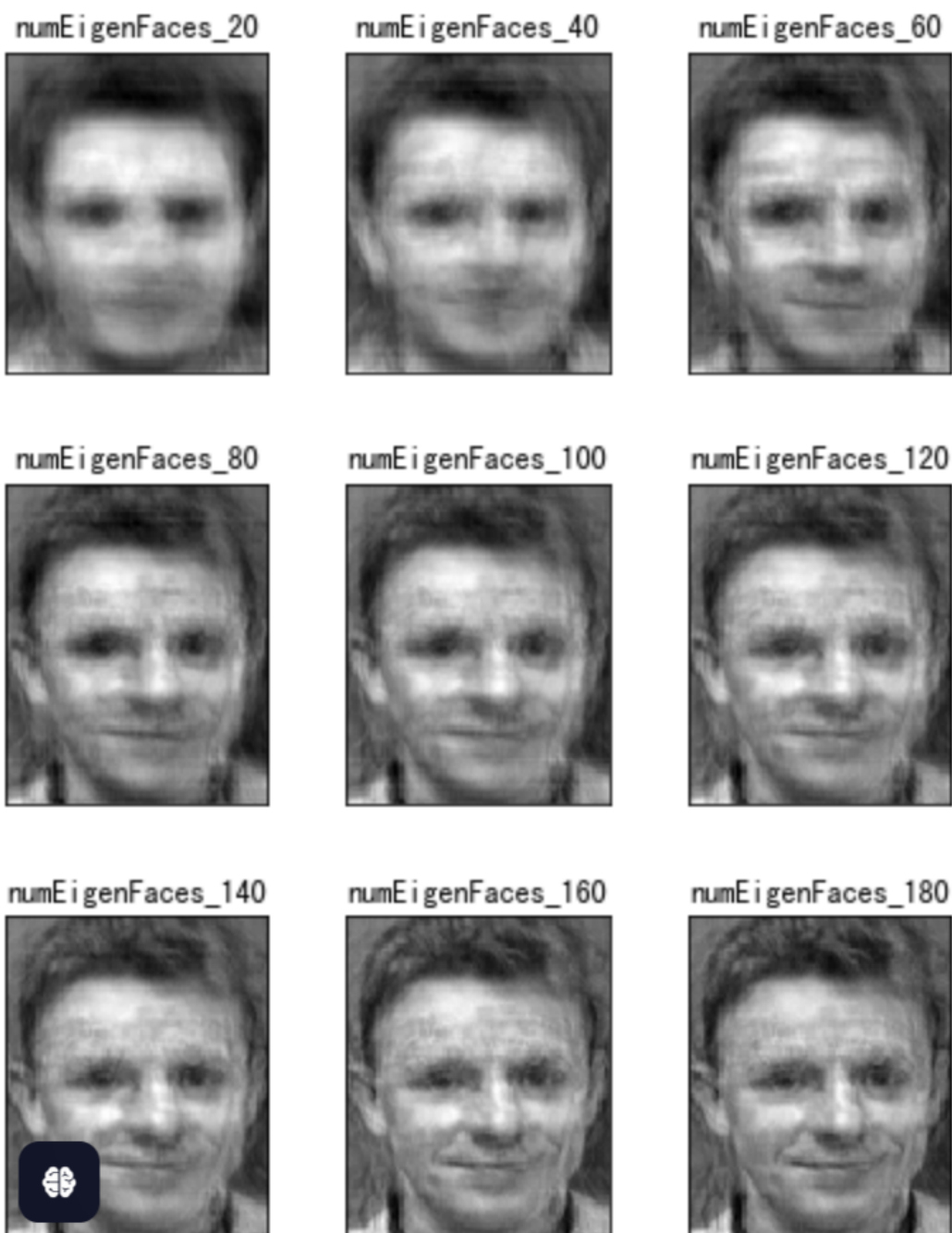
实验结果

发现得到的特征人脸比之前的更加抽象模糊。



人脸识别的正确率也只有**73%**，而且打印信息发现，大部分加入的样本的识别成功率都不高。看到这个结果就知道大概率是加入的样本的问题，一是没有多角度，二是人脸的大小和位置不对。

重建人脸，重建的是原本数据库中的，发现效果和原本的差不多。所以应该是加入的样本格式不正确导致后面的样本难以识别。



五、总结

- 在本次实验中，一方面复习了PCA和SVD奇异值分解的知识，熟悉了相关库的使用；另一方面，学习了特征人脸识别算法，熟悉了算法流程，并手动实现了算法。
- 尝试自己加入数据集，但是数据集真的不好找，找了两三个小时，很多数据集并没有同一样本下的多视角，甚至同样本的多图片都少见，有的数据集还得申请获取，不直接开源。再加上用cv库裁剪的效果并不好，模型的性能反而下降了。